

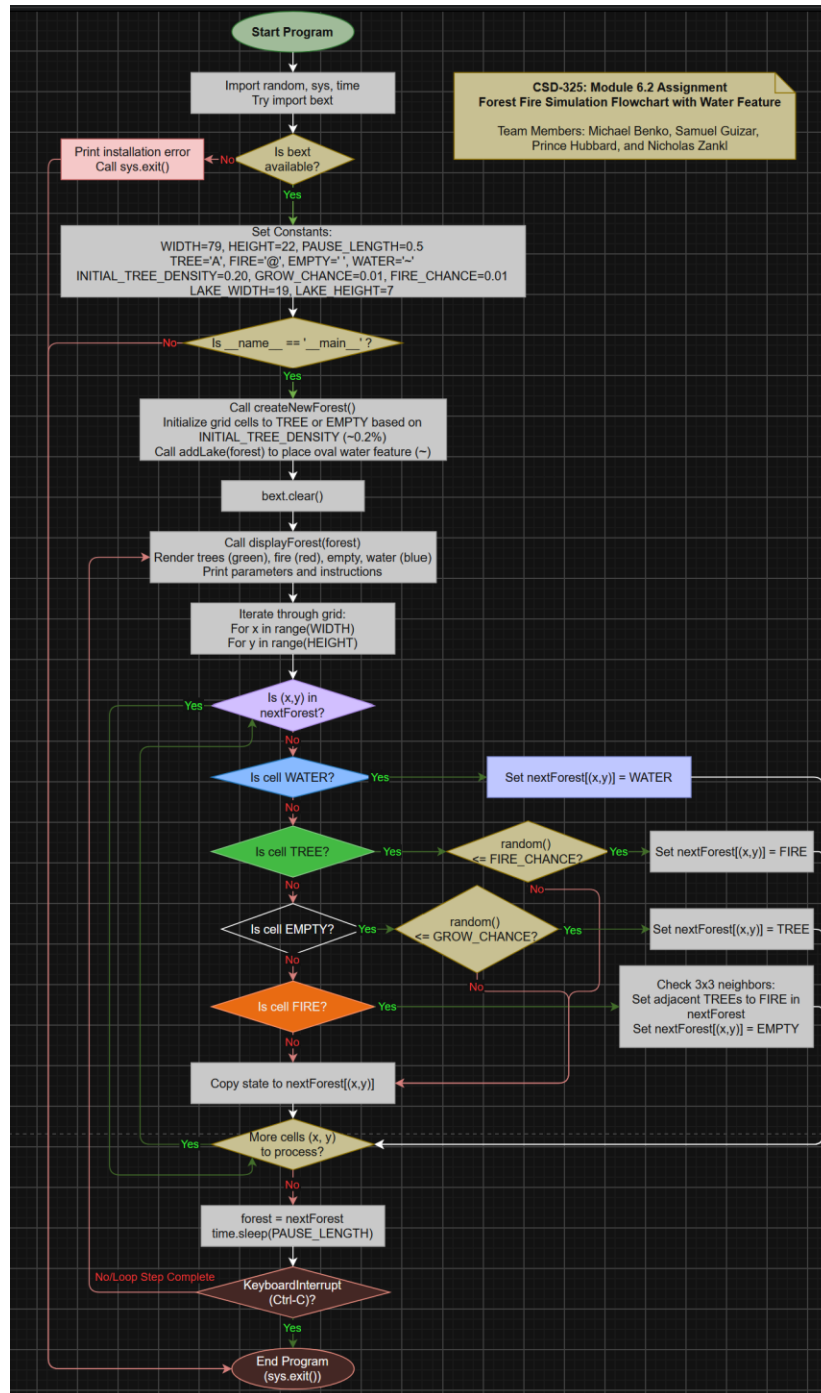
CSD-325: Module 6.2 Assignment

Forest Fire Simulation Flowchart

Date: September 13, 2026

Group C Members: Michael Benko, Samuel Guizar, Prince Hubbard, Nicholas Zankl

Figure 1: Revised Forest Fire Simulation Logic Flowchart



Description: A visual mapping of the updated execution flow demonstrating board setup, the priority evaluation of permanent water cells, probability checks for growth and fire, and loop termination upon user interrupt.

Introduction and Baseline Execution

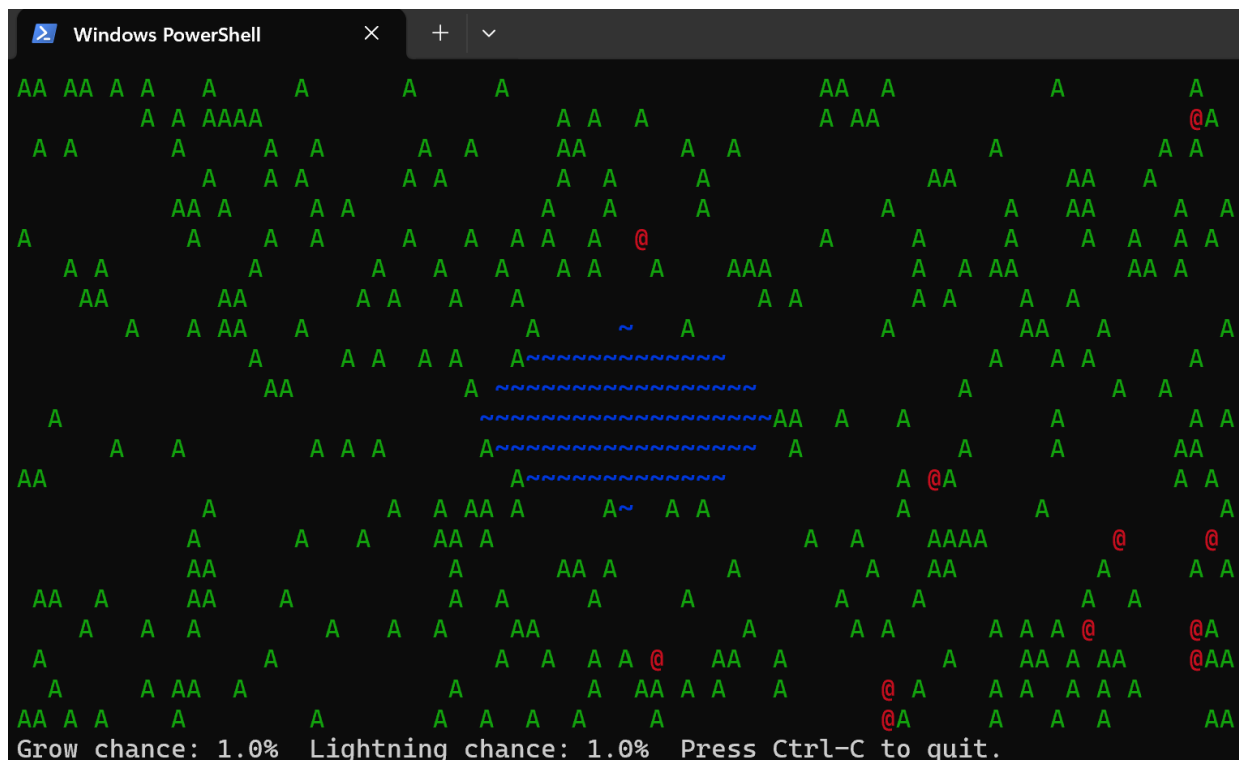
In Module 5.2, our team analyzed Al Sweigart's original Python implementation of the Forest Fire Simulation and created a structural flowchart detailing its execution flow, state transitions, and grid updating mechanism. The original program models dynamic wildfire behavior across a two-dimensional grid of 79 by 22 cells using discrete, frame-by-frame evaluations powered by Python's `bext` module. Execution begins by verifying dependencies and establishing global constants that define screen bounds, pause intervals, cell state characters, and probabilistic thresholds. Upon entering `main()`, `createNewForest()` populates the initial coordinate dictionary with trees and empty ground based on the initial density parameter, resulting in an actual starting tree density of approximately 0.2% across the grid. While `INITIAL_TREE_DENSITY = 0.20` was intended to yield a 20% tree density, combining a decimal constant with a 0–100 random scaling factor ($\times 100$) actually produces a 0.2% density upon execution. To maintain fidelity to the original codebase baseline, our team retained this legacy behavior for the Module 6.2 update while documenting the math discrepancy.

Once initialized, the terminal screen is cleared once via `bext.clear()` before entering the continuous simulation loop. On each frame pass, `displayForest()` repositions the cursor to (0, 0) and renders green trees, red fires, and blank spaces. The script then instantiates a double-buffered dictionary (`nextForest`) and iterates through every grid coordinate sequentially. To prevent double-processing within a single pass, coordinates already set in `nextForest` (such as those modified by neighboring fire spread) are immediately skipped. Unmodified cells fall into a cascading evaluation: tree cells check a 1% lightning chance to ignite, empty spaces check a 1% growth chance to sprout a tree, and burning fire cells check all eight surrounding neighbors to ignite adjacent trees before burning down to empty space. Any cell unaffected by these conditions falls through to a default state copy. Finally, forest updates to `nextForest`, execution pauses for 0.5 seconds, and the loop repeats until interrupted by Ctrl-C (`KeyboardInterrupt`). Establishing this clear control flow provided our team with the exact baseline required to implement the Module 6.2 environmental updates.

Technical Code Modifications & Flowchart Mapping (forestfiresim_325.py)

We modified the core codebase to incorporate a static, oval-shaped body of water near the center of the display grid. This update introduces a new environmental terrain element, a lake represented by the water symbol (~), that acts as an immutable firebreak to alter how fires propagate across the coordinate space. To establish this permanent body of water without hardcoding individual grid coordinates, we defined new global constants—WATER = '~', LAKE_WIDTH = 19, and LAKE_HEIGHT = 7—alongside the standard display dimensions. During board initialization inside createNewForest(), the program calculates initial tree density across the grid before passing the dictionary to addLake(forest). The addLake() function determines the center of the display grid using integer division on total width and height, establishing horizontal and vertical radii from the lake dimension constants. Using a normalized ellipse inequality formula, the function iterates across the bounding area and assigns the water character to every coordinate key falling within the calculated boundary.

Figure 2: Terminal Output of Forest Fire Simulation with Water Feature



Description: A screenshot of the live simulation running in Windows PowerShell, displaying the central blue water feature (~) functioning as a permanent firebreak surrounded by green tree cells (A) and active red wildfire cells (@).

Technical Code Modifications

During each step of the simulation loop in `main()`, the grid evaluates coordinate states sequentially. We updated the conditional logic tree to prioritize water coordinates before evaluating tree growth, lightning strikes, or fire spread. In the updated flowchart, immediately following the `nextForest` skip check, execution encounters the new `Is cell WATER?` decision node. On a positive result, execution immediately assigns water to `nextForest` and skips all remaining evaluations, creating a static guard that ensures the lake remains completely unalterable throughout the simulation. On a negative result, execution proceeds down into the original cascading decision nodes for tree cells, empty spaces, and active fires.

Pixel State Transition Rules

Pixel state transitions adhere strictly to this priority order on every frame pass. Water cells are preserved first as immutable terrain. Unmodified empty spaces then evaluate for a 1% growth chance to sprout a tree, while existing tree cells evaluate for a 1% lightning strike chance to ignite. Active fire cells check their eight surrounding neighbors to propagate fire exclusively to adjacent tree tiles before transitioning to empty ground. Because fire spread logic strictly targets tree characters, fire can neither ignite nor cross water cells, allowing the lake to act as an impassable barrier that forces wildland fires to burn around its perimeter. Finally, we modified `displayForest()` to set the terminal foreground color to blue whenever a water cell is printed, establishing a clear visual contrast between the blue lake, green trees, and red active wildfires in the terminal output.